

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-a9038905afcca0755.jsonl`
- SHA-256: `ceb011be2d5a2b18014797d73704202e24b7df5a5f4b073c2139c9b387fb8c16`
- Source modified: `2026-06-13T19:13:01+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:09:22.947Z)

Verify the correctness + scale/translation-invariance of
backend/pipeline/detectors/fusion_features.py: point_in_polygon (ray-casting edge cases, horizontal edges, vertices), foot/center normalization, in_court_counts, players_in_court (median), in_court_ratio, mean_player_motion (track matching across frames), two_sided_motion (net split), shuttle_player_colocation (frame alignment, pad, denominator = only aligned visible frames). Find any math bug, wrong edge case, or coordinate-space inconsistency (pixels vs normalized). Are all outputs in sane ranges?

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:25.897Z)

I'll start by reading the actual files involved in this change. Let me pull the latest state first and read the core files in parallel.

3. user (2026-06-13T19:09:26.798Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
```

```

timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
68         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
69         True (count everyone) when it is None."""
70         if court_polygon is None:

```

```

72         return True
73     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                         frame_height: float,
78                         court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82                 court_polygon))
83             for _frame_idx, dets in sorted(per_frame.items())
84         ]
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133         0.0.

```

```

133     """
134     if net is None or not per_frame:
135         return 0.0
136     axis, pos = net
137     both = 0
138     for _frame_idx, dets in per_frame.items():
139         near, far = False, False
140         for _tid, box in dets:
141             if not person_in_court(box, frame_width, frame_height, court_polygon):
142                 continue
143             fx, fy = _foot_norm(box, frame_width, frame_height)
144             coord = fx if axis == "x" else fy
145             if coord < pos:
146                 near = True
147             else:
148                 far = True
149         if near and far:
150             both += 1
151     return both / len(per_frame)
152
153
154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                        pad_frac: float) -> bool:
156         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157         width/height (the 'adjacent / in-hand' tolerance)?"""
158         x1, y1, x2, y2 = box
159         px = pad_frac * frame_width
160         py = pad_frac * frame_height
161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                  frame_width: float, frame_height: float,
166                                  pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169         ~0**
170         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171         """
172         if not per_frame or frame_width <= 0 or frame_height <= 0:
173             return 0.0
174         aligned = 0
175         colocated = 0
176         for p in shuttle_points or []:
177             if not getattr(p, "visible", False):
178                 continue
179             dets = per_frame.get(int(p.frame))
180             if dets is None:
181                 # shuttle frame wasn't person-sampled ? can't judge
182                 continue
183             aligned += 1
184             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
185                    for _tid, box in dets):
186                 colocated += 1
187         return colocated / aligned if aligned else 0.0
188
189     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
190     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
191     PERSON_FEATURE_NAMES = [
192         "players_in_court", "two_sided_motion", "mean_player_motion",
193         "in_court_ratio", "shuttle_player_colocation",
194     ]

```

```

195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201     """Aggregate the per-window person features from per-frame detections + the in-
window
202     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204     return {
205         "players_in_court": players_in_court(counts),
206         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208         "in_court_ratio": in_court_ratio(counts, min_players),
209         "shuttle_player_colocation": shuttle_player_colocation(
210             per_frame, shuttle_points, frame_width, frame_height),
211     }
212

```

4. user (2026-06-13T19:09:28.368Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index bb56881..7f8191c 100644
```

```
--- a/backend/config/models.py
```

```
+++ b/backend/config/models.py
```

```
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
```

```
    min_free_gb: float = 0.0
```

```
+class FusionConfig(BaseModel):
```

```
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
```

```
+    +
```

```
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
```

```
+    """
```

```
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
```

```
+    substrate: Literal["wasb", "tracknet"] = "wasb"
```

```
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
```

```
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
```

```
+    # enabled ? so the default path never imports torch / touches the GPU.
```

```
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
```

```
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
```

```
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
scoring).
```

```
+    min_crossings: int = Field(default=0, ge=0)
```

```
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
```

```
+    # < this. 0 = OFF.
```

```
+    min_players: int = Field(default=0, ge=0)
```

```
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
```

```
+    court_polygon: Optional[list[list[float]]] = None
```

```
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
```

```
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
```

```

+     net_axis: Literal["x", "y"] = "y"
+     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+
+ class IndexingConfig(BaseModel):
+     default_segmenter: str = "yolo_hybrid"
+     use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):
+
+     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
+     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+     fusion: FusionConfig = Field(default_factory=FusionConfig)
+
+     @field_validator("default_segmenter")
+     @classmethod
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
+     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
+
+     return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                          frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's ``.frame`` exactly ? that alignment is what
+     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break

```

```

+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+             frame_idx += 1
+         cap.release()
+         return {"frames": frames, "fps": fps,
+                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
+
+__all__ = [
+    "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
+    "HybridYoloSegmenter",
+    "TrackNetHybridSegmenter",
+    "WasbHybridSegmenter",
+    "FusionSegmenter",
+    ]
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+    except (ValueError, TypeError):
+        return max(3, int(duration / 0.8))

+
+    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
+    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                                frame_width: float, video_path: str,
+                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+        return {
+            "peak_velocity": cw["peak_velocity"],
+            "mean_velocity": cw["mean_velocity"],
+            "active_frame_count": cw["active_frame_count"],
+            "track_id_count": cw["track_id_count"],
+        }
+
+    def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                            window_stats: List[Dict[str, Any]],
+                            idx_cfg: Dict[str, Any]) -> List[int]:
+        """Indices of the candidate windows that proceed to the AI handoff (and thus may
+        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+        Persisted candidates are NOT affected ? they remain the full superset for offline
+        re-scoring."""
+        return list(range(len(windows)))
+
+    def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                      player_pool: Optional[List[str]] = None,
+                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
+        f"peak_v={cw['peak_velocity']:.3f}

```

```

mean_v={cw['mean_velocity']:.3f}")

+      # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+      # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+      # persistence and the handoff gate below.
+      window_stats = [
+          self.enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+          for cw in all_candidate_windows
+      ]
+
+      # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+      candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+      if store_candidates:
+          for i, cw in enumerate(all_candidate_windows):
-              stats = {
-                  "peak_velocity": cw["peak_velocity"],
-                  "mean_velocity": cw["mean_velocity"],
-                  "active_frame_count": cw["active_frame_count"],
-                  "track_id_count": cw["track_id_count"],
-              }
+              candidate_ids[i] = self.db.add_candidate_segment(
+                  video_id, detector=self.name,
+                  start_time=cw["start"], end_time=cw["end"],
+                  chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-                  stats=stats, detection_params=detection_params,
+                  stats=window_stats[i], detection_params=detection_params,
+              )

+      if not all_candidate_windows:
+          return []

+      # --- Phase 3: AI provider handoff over padded candidate clips ---
+      # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+      # candidates above stay the full superset ? only the served play_segments are gated.
+      handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
+      handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+      os.makedirs(handoff_dir, exist_ok=True)
+      logger.info("Phase 3: %s validation on %d candidate windows...",
-                  provider_name, len(all_candidate_windows))
+                  provider_name, len(handoff_indices))

+      def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+          w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

+      ai_segments: List[dict] = []
+      with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-          futures = [executor.submit(process_candidate_window, wi, cw)
-                      for wi, cw in enumerate(all_candidate_windows)]
+          futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                      for wi in handoff_indices]
+          for future in futures:
+              ai_segments.extend(future.result())
+      try:

```

5. assistant (2026-06-13T19:09:32.223Z)

Now let me read the new fusion_hybrid.py segmenter and the relevant parts of person_detector.py and trajectory_hybrid.py to understand the full context ? especially the shuttle point shape, the frame alignment, and how features get called.

6. user (2026-06-13T19:09:33.200Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

```



```

2
3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13 never imports torch / touches the GPU.
14 - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17 **Persisted candidates remain the full superset** regardless, so the offline
experiment
18 harness can re-score any variant.
19
20 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24 experiment harness (EXPERIMENT_HARNESS.md), never silently.
25 """
26 from typing import Any, Dict, List, Optional, Tuple
27
28 from backend.pipeline.segmenters.base import segmenter_registry
29 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors import fusion_features as ff
35
36
37 class FusionSegmenter(TrajectoryHybridSegmenter):
38     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40     family = "fusion"
41     display_label = "Fusion"
42
43     def __init__(self, db: Any, config: Any):
44         super().__init__(db, config)
45         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the ")

```

```

56         "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],

```

```

110             window_stats: List[Dict[str, Any]],
111             idx_cfg: Dict[str, Any]) -> List[int]:
112     """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113     (0) keep every window ? identical to the substrate. Persisted candidates are
114     untouched (full superset)."""
115     fcfg = idx_cfg.get("fusion", {})
116     min_crossings = int(fcfg.get("min_crossings", 0))
117     min_players = int(fcfg.get("min_players", 0))
118     person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120     keep: List[int] = []
121     for i, st in enumerate(window_stats):
122         if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123             continue
124         if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125             continue
126         keep.append(i)
127     return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

7. user (2026-06-13T19:09:33.449Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))

```

```

39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55     Any]]:
56         """Return (runner, detector-specific detection_params extras)."""
57         raise NotImplementedError
58
59     @staticmethod
60     def _probe_fps_width(video_path: str) -> tuple:
61         """Return (fps, frame_width) for a video, with sensible fallbacks."""
62         cap = cv2.VideoCapture(video_path)
63         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65         cap.release()
66         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68     @staticmethod
69     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70         """Provider-reported exchange count, else a duration-based estimate.
71
72         One canonical implementation ? the twins had drifted (wasb relied on
73         ``int(None)`` raising into the fallback; same result, by accident).
74         """
75         shot_count = segment.get("shuttle_exchange_count")
76         if shot_count is None:
77             return max(3, int(duration / 0.8))
78         try:
79             return int(shot_count)
80         except (ValueError, TypeError):
81             return max(3, int(duration / 0.8))
82
83     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84     -----
85     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86     frame_width: float, video_path: str,
87     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89         BASE
90         returns exactly the 4 motion keys, so the trajectory twins persist byte-
91         identical
92         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93         features). ``points`` are the whole-video trajectory points
94         (TrajectoryPoint)."""
95         return {
96             "peak_velocity": cw["peak_velocity"],
97             "mean_velocity": cw["mean_velocity"],
98             "active_frame_count": cw["active_frame_count"],
99             "track_id_count": cw["track_id_count"],
100         }
101
102     def _select_for_handoff(self, windows: List[Dict[str, Any]],
103     window_stats: List[Dict[str, Any]],

```

```

98             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                     player_pool: Optional[List[str]] = None,
108                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,

```

```

159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally]
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198 f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200 f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],
221                 chunk_index=cw["chunk_index"], confidence=min(1.0,
222                 cw["peak_velocity"]),
223                 stats=window_stats[i], detection_params=detection_params,

```

```

217         )
218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229                 provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                    for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])

```

```

274         dur = end - start
275         metadata = {
276             "shot_count": self._shot_count_from(segment, dur),
277             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278             "detector": f"{self.family}-hybrid-{model_name}",
279         }
280         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281         if segment_id:
282             candidate_id = segment.get("_candidate_id")
283             if candidate_id is not None:
284                 self.db.link_candidate_to_segment(candidate_id, segment_id)
285             segments_data.append({
286                 "id": segment_id, "start_time": start, "end_time": end,
287                 "duration": dur, "metadata": metadata,
288             })
289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

8. assistant (2026-06-13T19:09:36.646Z)

Now let me look at the shuttle point shape (TrajectoryPoint), the rally_gate net-crossing logic to compare coordinate spaces, and the PersonDetector's `detect\_and\_track` / `make\_tracker` to confirm the box format and track_id semantics.

9. user (2026-06-13T19:09:39.564Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool

```



```

35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93     """Annotate each (start,end) window with its shuttle net-crossing count and a
94     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95     axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96     axis, net, span = estimate_play_axis(points)
97     deadband = deadband_frac * span
98     out: List[GatedWindow] = []

```

```

99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

10. user (2026-06-13T19:09:39.646Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6     identically);
7     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8     later the learned classifier reads its persisted features. Keeping detection here (not
9     in
10    the segmenter) is what makes "add a cue = register a producer" true.
11
12    Behaviour is unchanged from the in-lined version ? only the home moved:
13    - `lazy_load_model` ? load a permissively-licensed torchvision detector
14    - `make_tracker` ? a fresh supervision ByteTrack per chunk
15    - `detect_and_track` ? one frame ? confident persons with persistent IDs
16    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
17      with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
18      that feed the candidate `stats` JSON and the learned fusion features.
19
20    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
21    importing
22    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
23    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
24    (ADR-005; ultralytics' AGPL YOLO was dropped).
25    """
26    import logging
27    from typing import Any, Dict, List, Optional, Tuple
28
29    import cv2
30    import torch
31
32    from backend.utils.geometry import get_velocity_normalised
33
34    logger = logging.getLogger(__name__)
35
36    # torchvision detection models are trained on COCO where the "person" category is
37    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
38    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
39    _PERSON_CLASS_ID = 1
40
41    # Permissively-licensed torchvision detectors selectable via config
42    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
43    _DETECTOR_FACTORIES = {
44        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
45        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
46        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
47    }

```

```

45     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48     class PersonDetector:
49         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53         min_action_window_duration}``); the model loads lazily on first use.
54         """
55
56         def __init__(self, config: Dict[str, Any]):
57             self.config = config
58             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59             # `self.model([tensor])` inference call type-checks without a quarantine.
60             self.model: Any = None
61             self.device: Optional[str] = None
62
63         def lazy_load_model(self) -> None:
64             if self.model is not None:
65                 return
66
67             # Imported lazily so the module imports without torchvision present and so
68             # the test suite (which pre-sets self.model) never pulls in model weights.
69             from torchvision.models import detection as tv_detection
70
71             idx_cfg = self.config.get("indexing", {})
72             use_gpu = idx_cfg.get("use_gpu", True)
73             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74             if use_gpu and not torch.cuda.is_available():
75                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78             if model_name not in _DETECTOR_FACTORIES:
79                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80                 model_name = _DEFAULT_DETECTOR
81
82             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85             self.model = builder(weights="DEFAULT")
86             self.model.eval()
87             self.model.to(self.device)
88
89         def make_tracker(self, fps: float) -> Any:
90             """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92             Imported lazily so tests can mock this method without supervision installed.
93             """
94             import supervision as sv
95             frame_rate = max(1, int(round(fps)))
96             return sv.ByteTrack(frame_rate=frame_rate)
97
98         def detect_and_track(self, frame: Any, tracker: Any,
99                             score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100             """Detect persons in a frame and assign persistent track IDs.
101
102             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103             This replaces the old ultralytics ``model.track(...)`` call, which bundled
104             detection AND association: torchvision does the detection, supervision's
105             ByteTrack does the association.
106             """

```

```

107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
112         1).float().div(255.0).to(self.device)
113         with torch.no_grad():
114             outputs = self.model([tensor])
115
116         out = outputs[0]
117         boxes = out["boxes"].detach().cpu().numpy()
118         labels = out["labels"].detach().cpu().numpy()
119         scores = out["scores"].detach().cpu().numpy()
120
121         # Keep only confident person detections.
122         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
123         if not mask.any():
124             return []
125
126         # ByteTrack requires confidence to be populated (it filters on it internally).
127         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
128         class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):

```

11. assistant (2026-06-13T19:09:43.747Z)

Now I have a clear picture. Let me verify a few critical details. First, the `TrajectoryPoint` shape (does `.frame`` align as integer?), and confirm the rest of ``detect_and_track``. Let me also check how ``gate_windows`` uses `min_crossings=0` in the fusion call (so it doesn't filter) and how ``_points_in`` differs from the fusion's own ``shuttle_in`` slice.

12. user (2026-06-13T19:09:45.185Z)

```

128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139     velocity_thresh: float, merge_gap: float,
140     frame_skip: int = 3,
141     chunk_index: Optional[int] = None) ->
142     List[Dict[str, Any]]:
143         """
144         Runs person detection + tracking on a video chunk and extracts high-motion
145         action windows. Returns a list of dicts in the full video's timeframe, each
146         with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149
150         Args:
151             frame_skip: Process every Nth frame (1 = every frame). Higher values
152             dramatically reduce inference cost with minimal quality loss.
153             chunk_index: Index of the source chunk (recorded on each window for
154             traceability).
155         """

```

13. user (2026-06-13T19:09:46.560Z)

backend\pipeline\detectors\base.py:150: videos that happen to share a filename cannot reuse each other's frames or CSV.

14. assistant (2026-06-13T19:09:49.164Z)

Let me find the actual TrajectoryPoint definition to confirm `.frame` is an int and the coordinate space (pixels vs normalized) of `.x/.y`.

15. user (2026-06-13T19:09:51.172Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-118-
disk guard so a long extraction doesn't fill the disk mid-flight.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-119-
"""
backend\pipeline\detectors\base.py:120:         try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-121-
res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk 'NR==2{{print $4}}'")
backend\pipeline\detectors\base.py-122-         except (subprocess.TimeoutExpired, OSError):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-123-
return None
backend\pipeline\detectors\base.py:124:         try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-125-
return int((res.stdout or "").strip()) / (1024 ** 3)
backend\pipeline\detectors\base.py-126-         except (ValueError, AttributeError):
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-133-
return
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-134-
quoted = " ".join(wsl_tilde_quote(p) for p in paths)
backend\pipeline\detectors\base.py:135:         try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py-136-
self._wsl_bash(f"rm -rf {quoted}")
backend\pipeline\detectors\base.py-137-         except (subprocess.TimeoutExpired, OSError) as e:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-69-             f"python -c \"import torch;
print('torch', torch.__version__, \"
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-70-             f\"'cuda',
torch.cuda.is_available())\"")
backend\pipeline\detectors\wasb_runner.py:71:         try:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-72-             res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py-73-         except FileNotFoundError:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-84-             dest =
f"{self.cfg.wsl_stage_dir}/{dest_name}"
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-85-             cmd = f"mkdir -p
{self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} {wsl_tilde_quote(dest)} && echo OK"
backend\pipeline\detectors\wasb_runner.py:86:         try:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-87-             res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py-88-         except subprocess.TimeoutExpired:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-119-             base =
os.path.basename(video_win_path)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-120-             stem, ext = os.path.splitext(base)
backend\pipeline\detectors\wasb_runner.py:121:         # Unique, content-derived key: same
filename + different bytes -> different key.
```

```

backend\pipeline\detectors\wasb_runner.py-122-         key = f"{stem}__{video_id[:12]}" if
video_id else stem
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-123-         wsl_out_dir =
to_wsl_mnt_path(output_win_dir)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-152-     )
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-153-     logger.info("Running WASB
inference on %s ...", base)
backend\pipeline\detectors\wasb_runner.py:154:     try:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-155-         res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py-156-     except subprocess.TimeoutExpired:
--
backend\pipeline\detectors\tracknet_runner.py-4-inside WSL2 (ADR-001 in docs\DECISIONS.md). The
flow is:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-5-
backend\pipeline\detectors\tracknet_runner.py:6:     1. Translate the Windows video path to its
WSL-visible form (and optionally
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-7-         stage it into the WSL filesystem
to avoid slow /mnt/c per-frame reads).
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-8-     2. Invoke ``src/predict.py`` inside
the TrackNetV4 conda env via ``wsl``.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-78-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-79-@dataclass
backend\pipeline\detectors\tracknet_runner.py:80:class TrajectoryPoint:
backend\pipeline\detectors\tracknet_runner.py:81:    frame: int
backend\pipeline\detectors\tracknet_runner.py:82:    visible: bool
backend\pipeline\detectors\tracknet_runner.py:83:    x: float
backend\pipeline\detectors\tracknet_runner.py:84:    y: float
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-85-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-86-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-96-
backend\pipeline\detectors\tracknet_runner.py-97-    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:98:        """Verify the WSL env is usable: conda
env exists, TF imports, GPU visible.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-99-

```

[Showing results with pagination = limit: 60]

16. assistant (2026-06-13T19:09:55.564Z)

Confirmed: `TrajectoryPoint` has `frame: int`, `visible: bool`, `x: float`, `y: float`. Now I need to confirm a critical coordinate-space question: are the shuttle `.x`/`.y` in pixels or normalized? The fusion_features docstring (line 14) says "pixels ? the WASB trajectory shape", and `\_shuttle\_in\_box` compares `sx, sy` against pixel box coords. But `rally\_gate` uses the same `.x`/`.y` for net estimation (which is coordinate-space agnostic since it's relative). Let me check the WASB/TrackNet CSV parsing to confirm the actual coordinate space of `.x`/`.y`.

17. user (2026-06-13T19:09:57.714Z)

```

C:\Users\avidu\Projects\badminton-highlight-

```

```

indexer\backend\pipeline\detectors\wasb_runner.py-17-from typing import Dict, Any, Optional,
Tuple
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-18-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-19-# Reuse the model-agnostic helpers ?
trajectory shape + windowing are identical.
backend\pipeline\detectors\wasb_runner.py:20:# parse_trajectory_csv /
trajectory_to_action_windows are @staticmethods on TrackNetRunner.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-21-from
backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-22-from backend.pipeline.detectors.base import
DetectorRunner, WslCommandMixin, wsl_tilde_quote
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-23-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-175-         return expected_csv_win
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-176-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-177-     # Trajectory parsing + windowing are
inherited from tracknet_runner (model-agnostic).
backend\pipeline\detectors\wasb_runner.py:178:     parse_trajectory_csv =
staticmethod(TrackNetRunner.parse_trajectory_csv)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-179-     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-230-     #
----- CSV -> windows
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-231-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-232-     @staticmethod
backend\pipeline\detectors\tracknet_runner.py:233:     def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:
backend\pipeline\detectors\tracknet_runner.py-234-         """Parse a TrackNet predict CSV
(columns: Frame,Visibility,X,Y)."""
backend\pipeline\detectors\tracknet_runner.py-235-         points: List[TrajectoryPoint] = []
backend\pipeline\detectors\tracknet_runner.py-236-         with open(csv_win_path, newline="") as
f:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-237-             reader = csv.DictReader(f)
backend\pipeline\detectors\tracknet_runner.py-238-             for row in reader:
backend\pipeline\detectors\tracknet_runner.py-239-                 try:
backend\pipeline\detectors\tracknet_runner.py:240-                     points.append(TrajectoryPoint(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-241-                         frame=int(row["Frame"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-242-                         visible=int(row["Visibility"]) == 1,
backend\pipeline\detectors\tracknet_runner.py:243-                         x=float(row["X"]),
backend\pipeline\detectors\tracknet_runner.py:244-                         y=float(row["Y"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-245-                     ))
backend\pipeline\detectors\tracknet_runner.py-246-             except (KeyError, ValueError):
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-247-                 continue
backend\pipeline\detectors\tracknet_runner.py:248:         points.sort(key=lambda p: p.frame)

```

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-249-         return points
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-250-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-251-         @staticmethod
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-262-         """Convert a shuttle
trajectory into candidate rally windows.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-263-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-264-         A frame is "active" when the
shuttle is visible AND its inter-frame
backend\pipeline\detectors\tracknet_runner.py:265:         displacement (normalised by frame
width, per second) exceeds
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-266-         ``velocity_thresh`` ? i.e. the
shuttle is in flight. Active frames within
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-267-         ``merge_gap`` seconds of each
other are grouped into one window; short
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-268-         windows are dropped. Mirrors
HybridYoloSegmenter's windowing so the dict
--
backend\pipeline\detectors\tracknet_runner.py-273-         if frame_width <= 0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-274-             frame_width = 1280.0
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-275-
backend\pipeline\detectors\tracknet_runner.py:276:         # Compute per-frame normalised
velocity from the last *visible* point.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-277-         active = [] # (frame_idx,
velocity)
backend\pipeline\detectors\tracknet_runner.py-278-         prev: Optional[TrajectoryPoint] = None
backend\pipeline\detectors\tracknet_runner.py-279-         for p in points:
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-284-             dframes = p.frame -
prev.frame
backend\pipeline\detectors\tracknet_runner.py-285-             if dframes > 0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-286-                 dist = ((p.x -
prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
backend\pipeline\detectors\tracknet_runner.py:287:                 velocity = (dist /
frame_width) * (fps / dframes)
backend\pipeline\detectors\tracknet_runner.py-288-                 if velocity >
velocity_thresh:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-289-                     active.append((p.frame, velocity))
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-290-                     prev = p

```

18. assistant (2026-06-13T19:10:03.697Z)

This is the critical confirmation. The shuttle trajectory ``x`/`y`` are **raw pixel coordinates** from the CSV (``x`=float(row["X"])`, ``y`=float(row["Y"])`). The windowing normalizes by dividing by ``frame_width``. The velocity computation at line 287 divides ``dist / frame_width``, confirming ``x`/`y`` are in the same pixel space as the source resolution that produced the CSV.

Now here is a critical coordinate-space question for ``shuttle_player_colocation``: it compares

shuttle `p.x, p.y` (pixels, in the WASB CSV's resolution) against person boxes (pixels, from cv2's `CAP\_PROP\_FRAME\_WIDTH`). Let me verify whether WASB runs on the original resolution or a downscaled proxy. The memory mentions a "proxy-index/4K-stitch recipe". Let me check the WASB runner for any resolution scaling.

19. user (2026-06-13T19:10:04.782Z)

```
90         return None
91     if res.returncode != 0 or "OK" not in (res.stdout or ""):
92         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
93         return None
94     return dest
95
96     def _stage_infer_script(self) -> bool:
97         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
98         src_mnt = to_wsl_mnt_path(_INFER_WIN)
99         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
100
101         res = self._wsl_bash(cmd)
102         return res.returncode == 0 and "OK" in (res.stdout or "")
103
104     def run_predict(self, video_win_path: str, output_win_dir: str,
105                     video_id: Optional[str] = None) -> Optional[str]:
106         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
107
108         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
109         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
110         can never reuse each other's staged frames, resumable cache, or CSV.
111
112         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
113         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
114         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
115         # the CSV landed inside WSL while Windows polled the repo-relative path.
116         output_win_dir = os.path.abspath(output_win_dir)
117         os.makedirs(output_win_dir, exist_ok=True)
118         # Normalize for cross-platform basename (tests use Windows paths on Linux).
119         video_win_path = str(video_win_path).replace("\\", "/")
120         base = os.path.basename(video_win_path)
121         stem, ext = os.path.splitext(base)
122         # Unique, content-derived key: same filename + different bytes -> different key.
123         key = f"{stem}__{video_id[:12]}" if video_id else stem
124         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
125         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
126
127         if not self._stage_infer_script():
128             logger.error("Could not stage wasb_infer.py into WSL.")
129             return None
130         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
131         if staged_video is None:
132             return None
133         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
134
135         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
136         # WSL stage filesystem is already low so the user can clean up before it fills.
137         if self.cfg.min_free_gb > 0:
138             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
139             if free is not None and free < self.cfg.min_free_gb:
140                 logger.warning(
141                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
142
143                 "may fill the disk ? consider running tools/cleanup_caches.py.",
144                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
145
146         cmd = (
```

```

145         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
146         f"python wasb_infer.py "
147         f"--video {wsl_tilde_quote(staged_video)} "
148         f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
149         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
150         f"--sport {shlex.quote(self.cfg.sport)} "
151         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}"
152     )
153     logger.info("Running WASB inference on %s ...", base)
154     try:
155         res = self._wsl_bash(cmd)
156     except subprocess.TimeoutExpired:
157         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
158         return None
159     if res.returncode != 0:
160         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
161         return None
162     if not os.path.exists(expected_csv_win):
163         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
164         return None
165     logger.info("WASB trajectory CSV: %s", expected_csv_win)
166
167     # Success ? the trajectory CSV is the durable output; the staged video copy and
168     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
169     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
170     # so the frame cache + detector cache remain for a resumable re-run.
171     if not self.cfg.keep_frames:
172         logger.info("Cache hygiene: removing staged video + frame cache for %s "
173                     "(set indexing.wasb.keep_frames=true to retain).", key)
174         self._wsl_rm_rf([frames_dir, staged_video])
175     return expected_csv_win
176
177     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
178     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
179     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

20. assistant (2026-06-13T19:10:09.942Z)

The WASB inference extracts frames from the staged video at its native resolution (no downscale flag passed), so the shuttle CSV `.x/.y` are in the original video's pixel space ? the same space cv2 reads boxes in. So the pixel-space comparison in `shuttle_player_colocation` is consistent **as long as WASB ran on the same resolution as the cv2 frames**. This is a known assumption; I'll note it but it's not a code bug.

Now let me focus on the math. Let me carefully verify the `point_in_polygon` ray-casting edge cases, then check whether tests exist for `fusion_features` to see what's already covered.

21. user (2026-06-13T19:10:11.003Z)

```

tests\test_fusion_features.py
tests\__pycache__\test_fusion_features.cpython-313-pytest-9.0.3.pyc
tests\test_fusion_hybrid.py
tests\__pycache__\test_fusion_hybrid.cpython-313-pytest-9.0.3.pyc

```

22. user (2026-06-13T19:10:11.706Z)

```

Found 9 files
tests\test_fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\person_detector.py

```

```
backend\pipeline\detectors\fusion_features.py
docs\QUALITY_ITERATIONS.md
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\EXPERIMENT_HARNESS.md
backend\pipeline\validators\relevance.py
```

23. user (2026-06-13T19:10:14.385Z)

```
1      """Pure-logic unit tests for the person-cue fusion features
(backend.pipeline.detectors.fusion_features).
2
3      No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path
and
4      edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5      """
6      from types import SimpleNamespace
7
8      import pytest
9
10     from backend.pipeline.detectors import fusion_features as ff
11
12
13     def _shuttle(frame, x, y, visible=True):
14         return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17     # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18     FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19     LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22     # ----- point_in_polygon
23
24     def test_point_in_polygon_inside_outside():
25         assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26         assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27         assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28         assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31     def test_point_in_polygon_degenerate():
32         assert ff.point_in_polygon((0.5, 0.5), None) is False
33         assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36     # ----- person_in_court
37
38     def test_person_in_court_uses_foot_point_and_polygon():
39         # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40         in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41         out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42         assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43         assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44         # None polygon => everyone counts
45         assert ff.person_in_court(out_box, 100, 100, None) is True
46         # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47         assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50     # ----- counts / ratios
51
```

```

52     def test_in_court_counts_and_players_and_ratio():
53         near = (10, 10, 30, 30)      # foot (0.2,0.3) in LEFT_HALF
54         far = (60, 10, 80, 30)      # foot (0.7,0.3) NOT in LEFT_HALF
55         per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56         # With LEFT_HALF mask: counts = [1, 1, 0, 2]
57         counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
58         assert counts == [1, 1, 0, 2]
59         assert ff.players_in_court(counts) == 1.0          # median of [0,1,1,2]
60         assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
61         assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
62         # No mask: far counts too -> [2,1,0,2]
63         assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
64         assert ff.players_in_court([]) == 0.0
65         assert ff.in_court_ratio([]) == 0.0
66
67
68     # ----- mean_player_motion
69
70     def test_mean_player_motion():
71         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
72         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74         # single frame -> 0; no shared track id -> 0
75         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79     # ----- two_sided_motion
80
81     def test_two_sided_motion():
82         net = ("y", 0.5)
83         near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
84         far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
85         both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
86         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87         one_sided = {0: [(1, near)], 1: [(2, near)]}
88         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0    # no net -> 0
90
91
92     # ----- shuttle_player_colocation
93
94     def test_shuttle_player_colocation():
95         box = (40, 40, 60, 60)
96         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99         # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
100         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101         # all-held -> 1.0
102         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109     # ----- aggregator

```

```

110
111     def test_compute_person_features_keys():
112         near = (10, 10, 30, 40)
113         far = (10, 60, 30, 90)
114         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115         shuttle = [_shuttle(0, 20, 25, True)]
116         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
117                                           court_polygon=None, net=("y", 0.5))
118         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
119         assert all(isinstance(v, float) for v in feats.values())
120         assert feats["two_sided_motion"] == pytest.approx(1.0) # both sides occupied every
frame
121
122
123     def test_compute_person_features_empty():
124         feats = ff.compute_person_features({}, [], 100, 100)
125         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
126         assert all(v == 0.0 for v in feats.values())
127

```

24. assistant (2026-06-13T19:10:17.148Z)

Now let me carefully analyze the math. Let me run the existing tests and also do my own targeted probes to verify the edge cases the task asks about, particularly the ray-casting on horizontal edges and vertices.

25. user (2026-06-13T19:10:19.662Z)

```

..... [100%]
9 passed in 0.06s

```

26. assistant (2026-06-13T19:10:22.804Z)

Now let me write a focused probe of the math edge cases the task specifically asks about: ray-casting on vertices/horizontal edges, the `two\_sided\_motion` boundary semantics, the `players\_in\_court` median for even counts, and the colocation denominator. I'll verify against expected ray-casting behavior.

27. user (2026-06-13T19:10:31.189Z)

```

=== point_in_polygon edge cases ===
vertex (0,0): True
vertex (1,1): False
on bottom edge (0.5,0.0): True
on top edge (0.5,1.0): False
on left edge (0.0,0.5): True
on right edge (1.0,0.5): False
tri centroid (0.5,0.3): True
tri outside (0.9,0.9): False
diamond center (0.5,0.5): True
diamond y=0.5 left (-0.1,0.5): False
diamond y=0.5 inside (0.3,0.5): True

```

28. assistant (2026-06-13T19:10:37.854Z)

The ray-casting is the standard, well-behaved PNPOLY algorithm. Boundary behavior is the usual "half-open" convention (consistent and correct ? no double-counting at the y=0.5 vertices of the diamond). This is fine for a court mask where points exactly on the boundary are measure-zero. No bug there.

Now let me look at the more subtle issues. Let me examine the `two\_sided\_motion` net split semantics combined with `\_foot\_norm` returning `(0,0)` on bad dims, and critically ? the ****coordinate-space mismatch in `two\_sided\_motion`****: it compares the normalized foot coord `fy` (which is `y2/frame\_height`) against `net\_position` (normalized 0-1). That's consistent. But let

me check the colocation alignment carefully against the `frame\_skip` and `int(p.frame)` rounding, and whether `mean\_player\_motion`'s normalization-by-width-only is documented vs implemented.

29. user (2026-06-13T19:10:48.518Z)

```
=== mean_player_motion normalization ===
vertical 10px on 100x200 (height-normalized y): 0.049999999999999996
=== two_sided_motion boundary ===
both feet >=0.5 => no near => 0.0: 0.0
=== colocation denominator ===
aligned denom counts only sampled+visible: 1.0
```

30. assistant (2026-06-13T19:10:55.844Z)

Confirmed several behaviors. Now let me check the most important potential real bug: the `mean\_player\_motion` docstring claims "normalised by the frame width (resolution-invariant)" but it actually uses `\_center\_norm` which divides x by width and y by height. For a non-square (16:9) frame, this means x and y use different scale factors ? the displacement magnitude is **anisotropic** and therefore **not** a simple rotation/translation-invariant Euclidean speed. Let me verify whether this is actually a scale-invariance problem or just a docstring mismatch, and check the other functions for the same width-vs-height consistency.

Let me also verify the critical `\_shuttle\_in\_box` pad asymmetry: pad uses `pad\_frac \* frame\_width` for x and `pad\_frac \* frame\_height` for y ? also anisotropic but that's a tolerance so it's defensible. Let me check whether there's an actual scale-invariance break and verify `players\_in\_court` returns the count median (which is fine but note: it's a float median ? for even frames it averages, giving e.g. 1.5).

31. user (2026-06-13T19:11:05.321Z)

```
=== scale invariance check (2x upscale should be identical) ===
1 {'players_in_court': 2.0, 'two_sided_motion': 1.0, 'mean_player_motion': 0.025,
  'in_court_ratio': 1.0, 'shuttle_player_colocation': 1.0}
2 {'players_in_court': 2.0, 'two_sided_motion': 1.0, 'mean_player_motion': 0.025,
  'in_court_ratio': 1.0, 'shuttle_player_colocation': 1.0}

=== aspect-ratio invariance: same normalized scene, different W:H ===
100x100: horiz0.1=0.1000 vert0.1=0.1000
160x90:  horiz0.1=0.1000 vert0.1=0.1000
```

32. assistant (2026-06-13T19:11:13.426Z)

Important finding: **uniform scale invariance holds** (2x upscale ? identical features), which is the property the module claims and the property that actually matters (same camera, different resolution). The aspect-ratio test shows that a 0.1-normalized move in x and a 0.1-normalized move in y both yield 0.1 because each axis is normalized by its own dimension ? so the docstring's "normalised by the frame width" is **inaccurate** (it's normalized per-axis by width AND height), but the actual behavior is **more** invariant than the docstring claims, not less. This is a docstring bug, not a math bug. I'll note it as low severity.

Now let me check a genuinely concerning area: the **frame alignment in `\_person\_features`** between the shuttle slice and the person detection window. The person detector reads frames `start\_f` to `end\_f` but only samples every `step` frames. The shuttle slice uses `start\_f` <= p.frame <= end_f. There's a subtle issue with how `detections\_per\_frame` computes `start\_frame` modulo: it samples `(frame\_idx - start\_frame) % step == 0`. Let me verify the colocation will actually find matches given that shuttle visible frames rarely land exactly on sampled person frames. Let me also check `frame\_skip` default consistency between the WASB substrate and person detector.

33. user (2026-06-13T19:11:13.912Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
```

```

3     Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4     keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5     substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6     persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7     """
8     from types import SimpleNamespace
9     from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
(~500)
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26         for f in range(300, 360):           # window B frames: y == net level -> deadband -> no
crossings
27             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28         return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")

```

```

56         return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
68         (superset)
69         assert all("net_crossings" in s for s in stats)
70         # Window A (rally) crosses many times; window B (held) zero.
71         assert stats[0]["net_crossings"] >= 2
72         assert stats[1]["net_crossings"] == 0.0
73         # Person cue OFF by default -> NONE of the person features present.
74         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
75         # The 4 base motion keys are still there.
76         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
77 "track_id_count"} <= set(s)
78         for s in stats)
79
80     def test_persisted_candidates_are_superset_of_substrate_windows():
81         db, _provider, _res = _run()
82         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
83 db.add_candidate_segment.call_args_list]
84         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
85         unfiltered
86
87     def test_default_hands_off_all_windows():
88         _db, provider, _res = _run()                            # min_crossings default 0 -> no
89         gating
90         assert provider.analyze_video.call_count == 2
91
92     def test_gate_a_drops_low_crossing_window_from_handoff_only():
93         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
94         # Persistence is still the full superset (offline substrate intact)...
95         assert len(_stats_calls(db)) == 2
96         # ...but only the high-crossing window A reaches the AI handoff.
97         assert provider.analyze_video.call_count == 1
98
99     def test_person_cue_adds_features_when_enabled():
100         fake_pd = MagicMock()
101         fake_pd.detections_per_frame.return_value = {
102             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
103             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
104         }
105         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
106 return_value=fake_pd):
107             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
108             stats = _stats_calls(db)
109             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
110             assert all("net_crossings" in s for s in stats)
111             fake_pd.detections_per_frame.assert_called()        # the person detector WAS used

```

34. assistant (2026-06-13T19:11:22.055Z)

Now I have all the context. Let me verify the critical frame-alignment bug I suspect in

`shuttle\_player\_colocation` combined with `detections\_per\_frame`. The person detector samples frames at `start\_frame + k\*step` (step=3 by default). The shuttle trajectory is dense (every frame). In `shuttle\_player\_colocation`, a shuttle point only counts toward `aligned` if `per\_frame.get(int(p.frame))` exists ? i.e., the shuttle frame index must exactly equal a sampled person frame. With step=3, only ~1/3 of shuttle frames align. That's by design (denominator = aligned frames), so it's not a bug ? but let me confirm the sampling offset is consistent and there's no systematic miss.

Let me verify one real concern: in `detections\_per\_frame`, `start\_frame = max(0, int(start\_frame))` but the shuttle slice in `\_person\_features` uses `start\_f = int(cw["start"] \* fps)` ? same computation, so frame indexing origin matches. Good. Let me run the full fusion test suite and do a final targeted check on the `two\_sided\_motion` denominator (it divides by `len(per\_frame)` which includes empty/dead frames ? is that the intended denominator?).

35. user (2026-06-13T19:11:32.114Z)

..... [100%]
14 passed in 6.34s

36. user (2026-06-13T19:11:43.214Z)

two_sided over 3 frames (1 empty): 0.6666666666666666
colocation guards W<=0: 0.0
median of [1,2] = 1.5
median of [0,1,2,3] = 1.5
in_court_ratio default min_players: 0.6666666666666666

37. assistant (2026-06-13T19:11:51.178Z)

All outputs are in sane ranges: ratios in [0,1], `players\_in\_court` non-negative (can be fractional 1.5 for even frame counts, which is fine for a learned feature), `mean\_player\_motion` non-negative.

Now let me check one last real concern: the `\_select\_for\_handoff` Gate B comparison `st.get("players\_in\_court", 0.0) < min\_players`. If the person cue is enabled and a window has all-dead frames, `players\_in\_court` could be 0.0 and get dropped, but `min\_players` is an int and `players\_in\_court` is a float ? comparison `1.5 < 2` works fine. But note: Gate B reads `players\_in\_court` from stats, which is only present when `person\_on` is True ? and the guard checks `person\_on` and `min\_players > 0` first, so no KeyError. Good.

Let me verify the one subtle correctness concern about `detections\_per\_frame` end-of-range: `while frame\_idx <= end\_frame` with `end\_f = int(cw["end"] \* fps)`. And confirm the `two\_sided\_motion` uses `person\_in\_court` (so off-court spectators don't create false two-sidedness) ? yes, line 141. Let me do a final check on the git diff of person_detector to confirm `Dict`/`Any`/`List`/`Tuple` are imported (the new method uses them).

38. user (2026-06-13T19:12:01.037Z)

module imports OK; typing names present: True
FusionConfig defaults: wasb 0.02 0 0 y 0.5 None {}
IndexingConfig has fusion: wasb

39. assistant (2026-06-13T19:12:05.804Z)

Everything imports and validates. Let me do one final verification of a potential coordinate-space inconsistency I want to be certain about: in `\_person\_features`, `fh = det["frame\_height"]` or 0.0. If `detections\_per\_frame` returns `frame\_height=0.0` (cv2 failed to read dims), then `fh=0.0`, and every feature that depends on height degrades to 0 (foot_norm returns (0,0), colocation guards on fh<=0). But `fw = det["frame\_width"]` or `frame\_width` falls back to the probed width while `fh` falls back to `**0.0**`, not a probed height. This is an asymmetry: width has a fallback, height does not. Let me confirm this is a real latent issue.

40. user (2026-06-13T19:12:10.710Z)

features with fh=0.0: {'players_in_court': 2.0, 'two_sided_motion': 0.0, 'mean_player_motion': 0.0, 'in_court_ratio': 1.0, 'shuttle_player_colocation': 0.0}

41. assistant (2026-06-13T19:12:23.023Z)

Confirmed. When `det["frame_height"]` is 0 (cv2 couldn't read dims, e.g. some codecs/containers), `fh=0.0` and three of five features silently collapse to 0 while `players_in_court/in_court_ratio` (which don't use height when polygon is None) still report normal values. This produces a misleading half-zeroed feature vector rather than an all-degraded one. The width path has a fallback (`det["frame_width"]` or `frame_width`); the height path does not. This is a real (low-severity, edge-case) inconsistency.

I now have a complete picture. Let me summarize my analysis of each item the task asked about:

****Verified CORRECT:****

- `point_in_polygon`: standard PNPOLY ray-casting. Horizontal edges correctly skipped via `(yi > y) != (yj > y)`. Vertices/boundary use a consistent half-open convention (no double-counting; verified on diamond). The `if yj != yi else xi` guard is dead-but-harmless (that branch is unreachable when `(yi>y)!= (yj>y)` is true, since that implies `yi != yj`).
- `_foot_norm/_center_norm`: correct bottom-center / center; guard `<=0` dims ? (0,0).
- `in_court_counts`: correctly sorted by frame, per-frame in-court count.
- `players_in_court`: correct median (fractional for even counts ? fine for a feature).
- `in_court_ratio`: correct fraction, [0,1], empty?0.
- `mean_player_motion`: track matching across consecutive sampled frames is correct; ****uniform-scale-invariant**** (verified 2x).
- `two_sided_motion`: net split correct; uses `person_in_court` so off-court persons don't create false two-sidedness; denominator = all sampled frames (intended); None net ? 0.
- `shuttle_player_colocation`: frame alignment correct (`int(p.frame)` vs integer keys); denominator = aligned visible frames only (correct as documented); pad anisotropy is a defensible tolerance.
- All outputs in sane ranges.

****Issues found**** (all low severity ? no high/med correctness bugs):

42. user (2026-06-13T19:12:49.816Z)

Structured output provided successfully

43. assistant (2026-06-13T19:13:01.325Z)

I've completed the verification of `backend/pipeline/detectors/fusion_features.py` and the surrounding fusion P2 wiring. Summary of what I found:

****No high- or medium-severity correctness bugs.**** The core math is sound:

- `point_in_polygon` is a correct PNPOLY ray-cast. Horizontal edges are correctly skipped via the `(yi > y) != (yj > y)` test, and vertices use a consistent half-open convention with no double-counting (verified on a diamond polygon where the ray passes through two y-equal vertices). The `if yj != yi else xi` branch is dead-but-harmless (unreachable when the crossing test is true).
- Foot/center normalization, `in_court_counts` (sorted by frame), `players_in_court` (median, fractional for even counts ? fine), `in_court_ratio`, `two_sided_motion` (net split using `person_in_court` so off-court spectators don't fake two-sidedness; denominator = all sampled frames), and `shuttle_player_colocation` (frame alignment via `int(p.frame)` vs integer keys; denominator = aligned visible frames only) are all correct.
- ****Uniform-scale invariance holds**** ? a 2x upscale of the whole scene gives byte-identical features. All outputs are in sane ranges (ratios in [0,1], counts/motion non-negative).
- All 23 fusion tests pass; config validates.

****Three low-severity issues reported:****

1. `fusion_features.py:108-109` ? `mean_player_motion` docstring claims "normalised by the frame width" but it actually normalises x by width and y by height (`_center_norm`). Docstring/spec mismatch, not a math bug; the real consequence is the feature is per-axis-normalised (not

rotation-isotropic), which the P3-classifier tuner should know.

2. ``fusion_hybrid.py:99-100`` ? asymmetric fallback: ``fw = det["frame_width"] or frame_width`` has a fallback but ``fh = det["frame_height"] or 0.0`` does not. If cv2 can't read the height, ``fh=0.0`` silently collapses the three height-dependent features to 0 while ``players_in_court`/`in_court_ratio`` still report normal values ? a misleading half-zeroed feature row that would poison the P3 training set.

3. ``fusion_features.py:154-185`` ? ``shuttle_player_colocation`` compares shuttle pixels (WASB CSV space) against person boxes (cv2 original-video space) with no scale conversion. Correct today because WASB infers on the full-res original, but latent breakage under the documented "proxy-index" recipe where WASB runs on a downscale. Worth an explicit invariant or normalizing both to `[0,1]` first.